WEEKLY INTERNSHIP REPORT

Week 1

Full Name	Muhammad Abdullah
Internship Domain	Frontend Development
Email Address	*m.abdullah98367@gmail.com*
Phone Number	*03062115084*

1. Overview

During Week 1 of the internship, I designed and built “Neon UI” — a reusable React component library. The goal of the project was to move away from writing repeated, one-off markup for common UI elements, and instead build a small set of configurable components (a Button, a Text Input, and a Profile Card) that can change their appearance and behavior purely through props, without editing their internal code. The project follows the Atomic Design methodology, where small, single-purpose components (atoms) are combined into larger, more complex pieces (molecules), which are then assembled into the full application (organism/page).

2. Objectives

- Build a Button component that supports multiple colors and sizes via props.
- Build a Text Input component that supports multiple colors and sizes via props.
- Build a Profile Card component that displays dynamic user data and follows a shared theme.
- Implement a global theme system so the entire UI updates when a color is selected.
- Demonstrate reusability by using each component multiple times with different props.
- Deploy the project and publish the source code to GitHub.

3. Work Completed

The following tasks were completed this week:

- Set up a new React project using Vite for fast local development and hot module reloading.
- Designed a dark, neon-themed visual identity using CSS custom properties (CSS variables) so that colors could be swapped globally without rewriting styles.
- Built the Buttons component (atom) with configurable color and size props, plus an active state for the theme switcher.
- Built the TextInput component (atom) with configurable size and, later, an optional color prop independent of the app theme.
- Built the ProfileCard component (molecule) combining an image, name, role, and a GitHub link into a single reusable card.
- Implemented a theme state in the main App component so that selecting a color button updates every themed component on the page at once.
- Added interactive behavior to the text input so a confirmation message only appears after the user finishes typing and presses Enter, rather than updating on every keystroke.
- Debugged and resolved import path errors and duplicate default export errors encountered during development.
- Initialized a Git repository, connected it to a remote GitHub repository, and pushed the completed project.
- Configured the project for deployment using GitHub Pages, making the working app viewable through a live link.

4. Code Section — Implementation Explanation

4.1 Reusable Button Component

The Button component accepts text, color, size, an active flag, and an onClick handler as props. Instead of writing a separate button for every color and size combination, the same component is reused with different prop values, and the CSS classes generated from those props determine its final appearance.

```
function Buttons({
  text,
  color = "blue",
  size = "medium",
  onClick,
  active = false
}) {
  return (
    <button
      className={`neon-button ${color} ${size} ${
        active ? "active" : ""
      }`}
      onClick={onClick}
    >
      <span>{text}</span>
    </button>
  );
}
```

This same component is used four times in the main app, once per theme color, proving that no duplicate button code was needed to support the different colors.

4.2 Reusable Text Input Component

The TextInput component accepts placeholder, value, onChange, onKeyDown, size, and an optional color prop. When no color is passed, the input automatically follows the app's current theme through CSS variables. When a color is explicitly passed, it overrides the theme and uses a fixed color instead — demonstrating that the same component can behave two different ways purely through props.

```
function TextInput({
  placeholder,
  value,
  onChange,
  onKeyDown,
  size = "medium",
  color
}) {
  return (
    <input
      type="text"
      className={`neon-input ${color ? color : ""} ${size}`}
      placeholder={placeholder}
      value={value}
      onChange={onChange}
      onKeyDown={onKeyDown}
    />
  );
}
```

In the main app, a small piece of state tracks what the user is currently typing, and a separate piece of state only updates when the Enter key is pressed. This means the confirmation message (“You have written: ...”) does not update on every keystroke — it waits until the user has finished typing.

```
const [username, setUsername] = useState("");
const [submittedName, setSubmittedName] = useState("");

function handleKeyDown(event) {
  if (event.key === "Enter" && username.trim() !== "") {
    setSubmittedName(username.trim());
  }
}
```

4.3 Reusable Profile Card Component

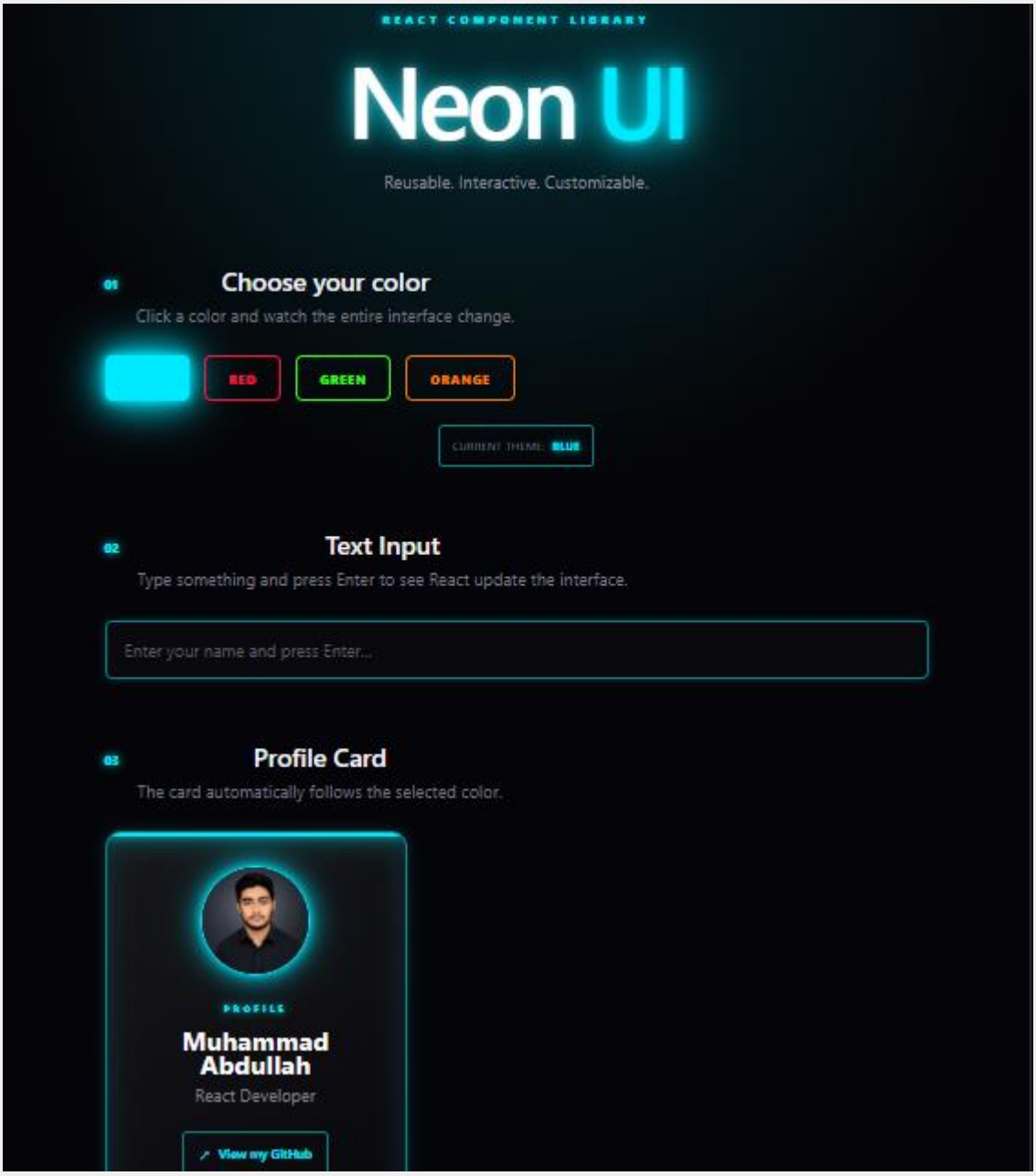
The ProfileCard component combines several smaller UI pieces — an image, a name, a role, and a GitHub link — into one reusable unit (a molecule, in Atomic Design terms). It accepts name, role, image, github, and theme as props, so a single component definition can represent any person's profile simply by passing different data.

```
function ProfileCard({ name, role, image, github, theme }) {
  return (
    <div className={`profile-card ${theme}`}>
      <div className="profile-image-wrapper">
        <img src={image} alt={name} className="profile-image" />
      </div>
      <div className="profile-info">
        <h3>{name}</h3>
        <p className="profile-role">{role}</p>
        <a href={github} target="_blank" rel="noopener noreferrer">
          View my GitHub
        </a>
      </div>
    </div>
  );
}
```

This component is used twice in the app with entirely different data — once for the main profile and once for a sample teammate — confirming that it is a generic, reusable card rather than a one-off block of hardcoded markup.

5. Screenshots — Project & Results

Application running locally, showing the default (blue) theme:



[Insert screenshot here: App running — Blue theme, all three components visible]

Buttons Code — selecting a different color button updates the entire interface:

App.jsx M | vite.config.js M | {} package.json M | # App.css | # index.css

src > components > Buttons.jsx > default

```jsx
function Buttons({
  text,
  color = "blue",
  size = "medium",
  onClick,
  active = false
}) {
  return (
    <button
      className={`neon-button ${color} ${size} ${
        active ? "active" : ""
      }`}
      onClick={onClick}
    >
      <span>{text}</span>
    </button>
  );
}

export default Buttons;
```

Butons.jsx file

Profile Card Code — confirmation message appearing after pressing Enter:

[Insert

App.jsx M | vite.config.js M | {} package.json M | # App.css | # index.css

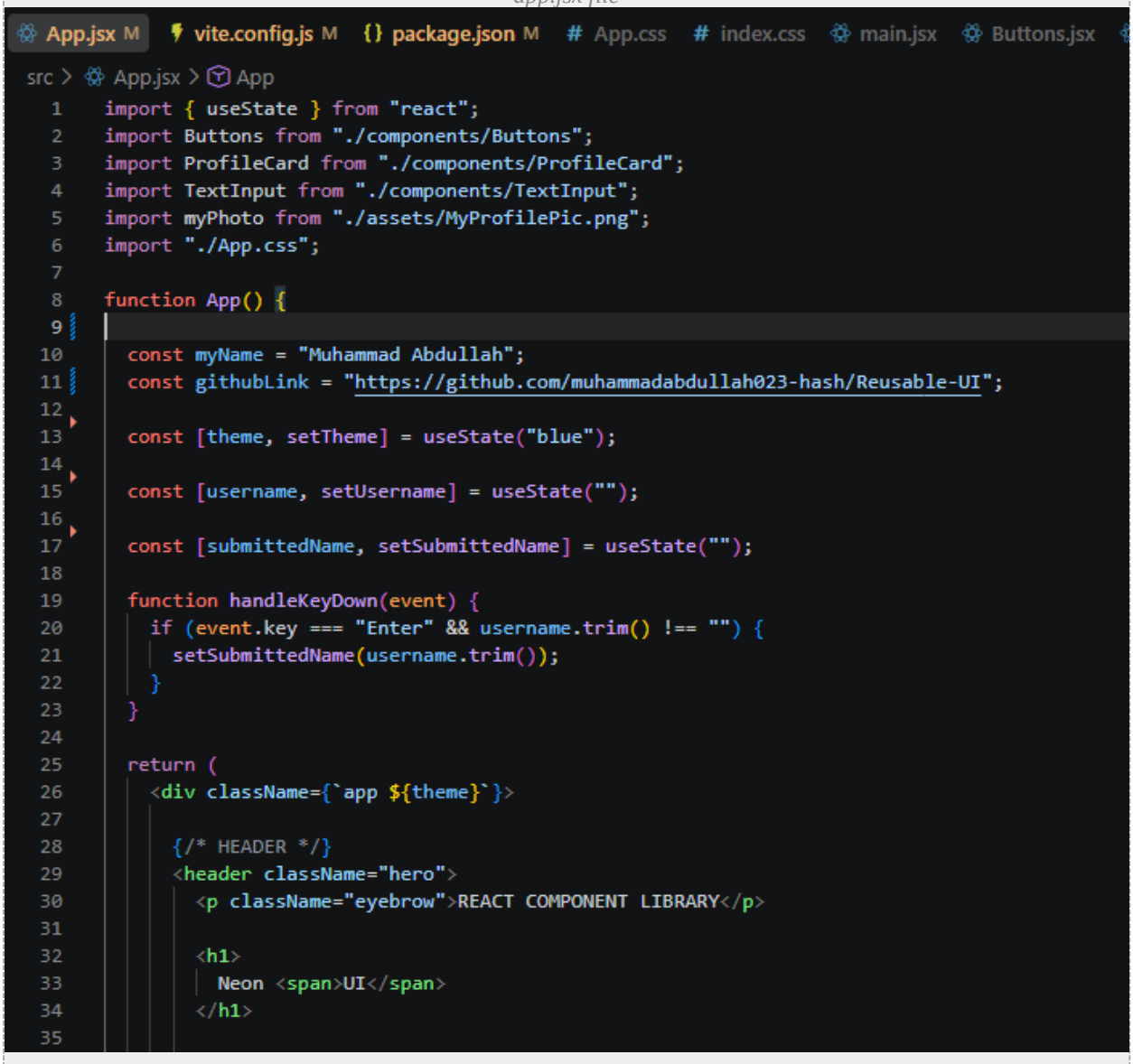
src > components > ProfileCard.jsx > ProfileCard

```jsx
function ProfileCard({
  name,
  role,
  image,
  github,
  theme
}) {
  return (
    <div className={`profile-card ${theme}`}>
      <div className="profile-glow"></div>
      <div className="profile-image-wrapper">
        <img
          src={image}
          alt={name}
          className="profile-image"
        />
      </div>
      <div className="profile-info">
        <p className="profile-label">
          PROFILE
        </p>
        <h3>{name}</h3>
        <p className="profile-role">
          {role}
        </p>
        <a
          href={github}
          target="_blank"
          rel="noopener noreferrer"
          className="github-link"
        >
          <span>↗</span>
          View my GitHub
        </a>
      </div>
    </div>
  );
}
export default ProfileCard;
```

ProfileCard.jsx file

App.jsx Code:

app.jsx file



```jsx
import { useState } from "react";
import Buttons from "./components/Buttons";
import ProfileCard from "./components/ProfileCard";
import TextInput from "./components/TextInput";
import myPhoto from "./assets/MyProfilePic.png";
import "./App.css";

function App() {

  const myName = "Muhammad Abdullah";
  const githubLink = "https://github.com/muhammadabdullah023-hash/Reusable-UI";

  const [theme, setTheme] = useState("blue");

  const [username, setUsername] = useState("");

  const [submittedName, setSubmittedName] = useState("");

  function handleKeyDown(event) {
    if (event.key === "Enter" && username.trim() !== "") {
      setSubmittedName(username.trim());
    }
  }

  return (
    <div className={`app ${theme}`}>

      {/* HEADER */}
      <header className="hero">
        <p className="eyebrow">REACT COMPONENT LIBRARY</p>

        <h1>
          Neon <span>UI</span>
        </h1>

```

6. Challenges Faced

- Resolved an import path mismatch where a component file name did not match the import statement, causing a Vite build failure.
- Fixed a duplicate default export error caused by two versions of a file being merged together during editing.
- Corrected Git configuration mistakes (using an invalid config key and an accidentally malformed remote URL) while setting up the GitHub repository.

7. Learnings

- Gained practical understanding of the Atomic Design pattern and how it applies to real component structuring.
- Learned how to drive component appearance entirely through props instead of hardcoding styles per instance.
- Learned Git fundamentals: initializing a repository, committing, connecting a remote, and pushing to GitHub.
- Learned how to deploy a Vite + React project using GitHub Pages.

8. Conclusion

By the end of Week 1, a fully functional, themeable, reusable UI component library was built, debugged, and published to GitHub. The project meets the core objective of demonstrating reusable UI blocks: each component — Button, Text Input, and Profile Card — can change its color and size without any changes to its underlying code, and each is reused multiple times throughout the application with different props to prove this in practice.

9. Project Link

GitHub Repository: https://github.com/muhammadabdullah023-hash/Reusable-UI